Assignment-6

Git Commit Requirements

Your Git history should show the progression of your work, not just the final solution.

What is required

- Make a commit after each meaningful change or milestone (creating a class header or source file, implementing a substantial class method, completing one part of the assignment, adding a new feature, or fixing a significant bug).
- Each commit message must briefly explain what you implemented, fixed, or changed.
- Commit regularly while working rather than committing the entire assignment at the end.
- Create commits as you work. Do not complete the assignment first and then divide the finished code into several commits afterward.
- You do not need to push after every commit. You may commit your changes locally and push all commits later.

Good commit examples

- `Implemented input validation in main function`
- `Fixed incorrect calculation in method getValue`
- `Updated Car class constructor`
- `Corrected output formatting for method print`
- `Implemented toString method in House.cpp`

Not meaningful

- `commit 1`
- `update`
- `test`
- `done`
- `final`
- Multiple commits that contain no meaningful change

Grading

Git-related deductions are not cumulative. The highest applicable deduction will be used.

- **Up to -20% — Poor commit quality**: You made multiple commits, but the messages are vague, repetitive, or do not clearly describe your changes, or your commits do not consistently represent meaningful progress.
- **Up to −40% — Insufficient development history:** Your Git history does not reasonably demonstrate incremental development. Examples include submitting most or all of the assignment in one or a few very large commits, making nearly all significant commits immediately before the deadline, creating several commits only after the code was already completed, or committing a large amount of new code only a few minutes after the previous commit.

A large number of commits by itself does not satisfy this requirement. Your commits should reflect actual, meaningful progress in your code.

Problem-1 (15 points):

Enhance the MyVector class by implementing a set of relational and equality operators, along with concatenation and output streaming.

Implement the following operators according to the exact logic described below:

1) **Equality** (== and !=)
   Two vectors are equal if and only if:
   - They have the same size, and
   - All corresponding elements are equal.
2) **Relational Operators** (<, >, <=, >=)
   Use lexicographical comparison with size precedence:
   - First compare sizes:
     - If this->size > other.size → *this > other
     - If this->size < other.size → *this < other
   - If sizes are equal, compare elements from left to right:
     - Return true for > if first mismatched element in *this is greater
     - Return true for < if first mismatched element in *this is smaller
     - If all elements are equal → vectors are equal → < and > return false
3) **Concatenation** (+)
   Returns a new vector that contains:
   - All elements of *this, followed by
   - All elements of other
4) **Assignment** (=)
   Perform a deep copy:
   - Delete current dynamic array
   - Copy size, capacity, and all elements from other
   - Allocate new memory and copy each element
5) **Output Stream** (<<)
   Print the vector with a space after each element and a newline at the end.

   [elem0 elem1 elem2 ...]

Example behavior:

```
a = [1, 2, 3]
b = [1, 2, 3, 4]
c = [1, 2, 4]

a == b    → false
a == [1,2,3] → true
a < b     → true (same prefix, but a is shorter)
b > a     → true
a > c     → false (at position 2: 3 > 4? no)
c > a     → true (at position 2: 4 > 3)
a + b     → [1,2,3,1,2,3,4]
```

Run main.cpp to test your code.

Problem-2 (15 points):

Enhance Hand class from Homework 4.

Add a static variable:

```
static int hands_dealt;
```

- Initialize it to 0 in the Hand.cpp file.
- Increment it every time a hand is created or re-dealt.

Update the constructor:

- Deal up to 5 cards from the deck (only if cards remain).
- Increment hands_dealt.

Implement new methods:

```
void getNewHand(Deck &deck);
int getHandsDealt();
bool fullHand() const;
```

- getNewHand() clears the hand and deals 5 new cards.
- getHandsDealt() returns the total number of hands dealt.
- fullHand() returns true if hand has exactly 5 cards and false otherwise.

Implement the following hand-checking functions:

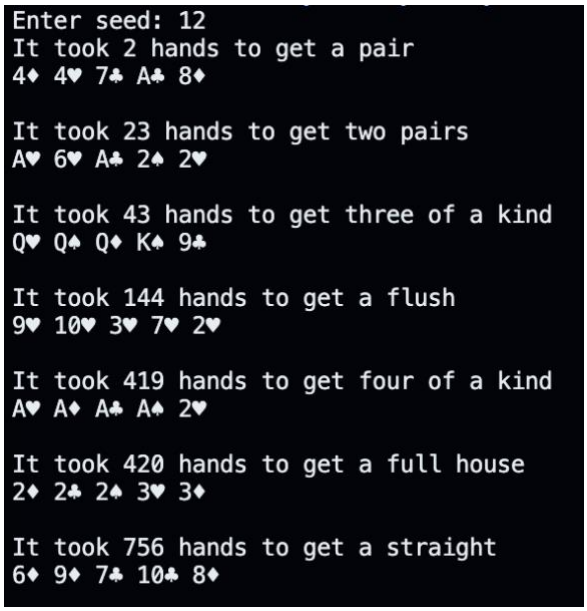
- `bool fourOfAKind() const;`
- `bool flush() const;`
- `bool straight() const;`
- `bool fullHouse() const;`

To recognize the appropriate poker hands:

- Four of a Kind
- Full House
- Flush
- Straight

Run main.cpp to test the code. Some seeds can lead to infinite loops.

Here is a sample run:



Problem-3 (5 points):

Design a generic function that returns true if **all** the elements in the second array are greater than **any** element in the first array.
Function has the following prototype:

```
template<typename T>
bool isGreater(const T* list1, const T* list2, int size1, int size2);
```